All `text` fields go through this pipe: raw text is cleaned by character filters, and the resulting text is passed on to the tokenizer. The tokenizer splits the text into tokens (individual words). The tokens then pass through token filters, where they are modified, enriched, and enhanced. The finalized tokens are stored in the appropriate inverted indexes. Search queries are analyzed the same way.

Figure 7.2 shows an example of the analysis process. We saw this figure in chapter 3, and it is presented again here for completeness.

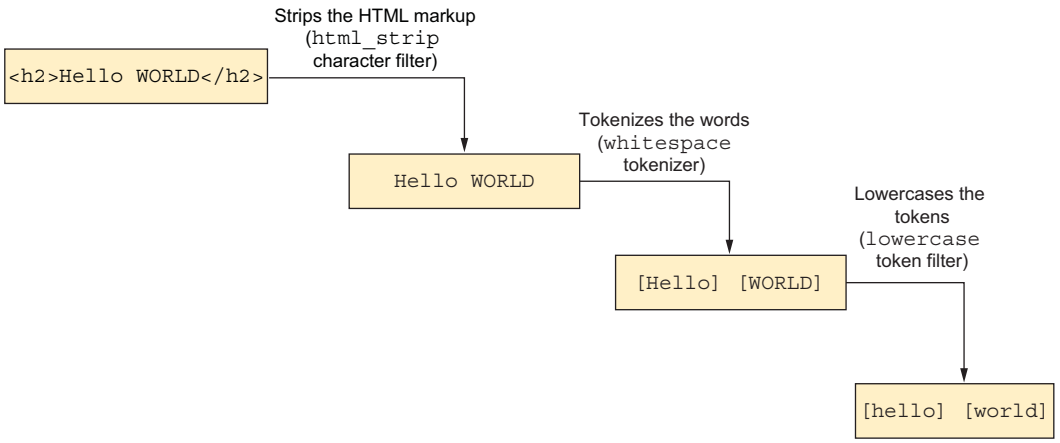


Figure 7.2 An example of text analysis in action

As we've mentioned, an analyzer is composed of three low-level building blocks:

- *Character filters*—Applied on the character level. Every character of the text goes through these filters. The filter's job is to remove unwanted characters from the text string. For example, this process can purge HTML tags like `<h1>`, `<href>`, and `<src>` from the input text. It also helps replace text with other text (e.g., Greek letters with the equivalent English words) or match text in a regular expression (regex) and replace it with its equivalent (e.g., match an email based on a regex and extract the domain of the organization). Character filters are optional; an analyzer can exist without a character filter. Elasticsearch provides three character filters out of the box: `html_strip`, `mapping`, and `pattern_replace`.
- *Tokenizers*—Split `text` fields into words using a delimiter such as whitespace, punctuation, or word boundaries. Every analyzer must have *one and only one* tokenizer. Elasticsearch provides a handful of tokenizers to help split incoming text into individual tokens that are then fed through the token filters for further normalization. Elasticsearch uses the `standard` tokenizer by default; it breaks words based on grammar and punctuation.